

ТОО «КОЛЛЕДЖ ХЕКСЛЕТ КАЗАХСТАН»

Отделение информационных технологий и программирования
Специальность 4S06130103 «Разработчик программного обеспечения»

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

К занятию 02. «Окружение и структура проекта»

Вариант 5: REST API-сервис
(код, миграции БД, тесты, документация)

Дисциплина: «Применение информационно-коммуникационных и цифровых технологий»

Выполнила:

студентка группы 21WD
Пилипенко Алина

Проверил:

преподаватель ИКТ, магистр управления и ИБ
Калиаскаров Д.А.

Оценка:

/ 100 б.

Подпись:

Алматы, 2026 г.

Цель и задачи работы

Цель: освоить архитектурные основы программного окружения, принципы взаимодействия операционной системы, прикладного ПО и файловой системы; спроектировать безопасную, масштабируемую и переносимую структуру проекта; исключить платформозависимые коллизии («работает только у меня») методами контейнеризации и стандартизации именования.

Задачи:

1. Спроектировать дерево каталогов REST API-сервиса (латиница, нижний регистр, без пробелов).
2. Обосновать выбор операционной системы для разработки и для production-сервера.
3. Разобрать 3 пути внутри проекта (1 абсолютный и 2 относительных).
4. Проанализировать и исправить 3 некорректных имени файлов под архитектуру REST API.
5. Описать конфигурацию воспроизводимой среды команды (Docker / WSL2).

1. Проектирование структуры папок проекта (Вариант 5)

Для REST API-сервиса спроектирована модульная архитектура, базирующаяся на принципах *Clean Architecture*, отраслевом манифесте *12-Factor App* и стандартах безопасной разработки.

Имена каталогов и файлов строго следуют правилам надежности:

- исключительно символы латинского алфавита;
- нижний регистр символов (*lowercase*);
- полное отсутствие пробелов и специальных символов;
- разделение смысловых блоков через дефис (*kebab-case*) для конфигураций или подчёркивание (*snake_case*) для Python-модулей.

```
rest-api-service/
├── app/
│   ├── api/
│   │   └── v1/
│   │       ├── auth.py
│   │       └── users.py
│   ├── core/
│   │   ├── config.py
│   │   └── security.py
│   ├── models/
│   │   └── user.py
│   ├── schemas/
│   │   └── user.py
│   ├── services/
│   │   └── user_service.py
│   ├── __init__.py
│   └── main.py
├── data/
│   └── initial_data.csv
├── docs/
│   ├── api.md
│   └── openapi.json
├── migrations/
│   ├── versions/
│   │   └── 001_initial_tables.py
│   ├── alembic.ini
│   └── env.py
├── tests/
│   ├── conftest.py
│   ├── test_api.py
│   └── test_auth.py
├── .dockerignore
├── .env.example
├── .gitignore
└── docker-compose.yml

# Исходный код сервиса (FastAPI / SQLAlchemy)
# Маршрутизация и HTTP-контроллеры
# Версионирование API v1
# Регистрация, авторизация, выпуск токенов
# CRUD операции над пользователями
# Ядро приложения и глобальные настройки
# Валидация переменных .env (Pydantic Settings)
# Хэширование паролей (bcrypt), JWT-токены
# Таблицы БД (SQLAlchemy ORM)
# Модель сущности пользователя
# Pydantic DTO (валидация входных/выходных данных)
# Схемы запросов и ответов
# Слой бизнес-логики (изоляция от базы данных)
# Обработка пользовательских сценариев
# Точка входа приложения (ASGI instance)
# Фикстуры и начальные справочники
# Данные для первоначального наполнения БД (seeding)
# Проектная документация
# Спецификация сценариев интеграции
# Машиночитаемая схема OpenAPI / Swagger
# Система версионирования схемы БД (Alembic)
# Атомарные скрипты миграций
# Конфигурационный файл Alembic
# Скрипт подключения Alembic к метаданным моделей
# Среда автоматизированного тестирования (pytest)
# Изолированные фикстуры и тестовая in-memory БД
# Интеграционные тесты эндпоинтов
# Модульные тесты механизмов безопасности
# Исключение лишних файлов из сборки образа Docker
# Публичный шаблон переменных окружения (без секретов)
# Правила игнорирования Git (.env, кэш, .venv)
# Оркестрация сервиса и СУБД PostgreSQL
```

└─ Dockerfile	# Спецификация сборки рабочего контейнера сервиса
└─ pyproject.toml	# Декларативный манифест зависимостей и сборщика
└─ README.md	# Руководство по разворачиванию, сборке и запуску

Функциональное назначение элементов:

- **app/** — главный пакет приложения. Разделение на слои `core/`, `api/`, `models/`, `schemas/` и `services/` гарантирует слабую связанность (*Loose Coupling*), защищает логику приложения от внешних воздействий и обеспечивает возможность изолированного тестирования.
- **migrations/** — версионирование схемы реляционной базы данных (Alembic). Исключает рассинхронизацию структуры таблиц между разными членами команды разработчиков.
- **tests/** — автоматические тесты (pytest). Позволяют мгновенно верифицировать корректность работы бизнес-логики перед слиянием кода в основную ветку.
- **docs/** — документация API (OpenAPI/Swagger), необходимая для прозрачной интеграции фронтенда и мобильных клиентов с серверной частью.
- **data/** — статические наборы данных (`initial_data.csv`) для безопасного наполнения тестовой или локальной базы данных начальными каталогами.
- **.env.example** и **.gitignore** — защита конфиденциальных данных. В репозиторий отправляется исключительно публичный шаблон настроек, тогда как рабочий файл с паролями `.env` изолируется от системы контроля версий.
- **Dockerfile** и **docker-compose.yml** — декларативное описание изолированной среды разворачивания REST API и СУБД PostgreSQL.

2. Обоснование выбора операционных систем

ОС для разработки (Workstation): Выбрана операционная система **Windows 11 с установленной подсистемой WSL2 (Ubuntu 24.04 LTS)**. Данная конфигурация позволяет разработчику комфортно использовать развитую графическую оболочку Windows, современные редакторы кода (VS Code, JetBrains) и инструменты тестирования API (Postman), одновременно получая доступ к полноценному нативному ядру Linux с идентичной файловой системой, что предотвращает появление скрытых платформенных ошибок.

ОС для сервера (Production): Выбрана серверная операционная система **Linux (Ubuntu Server 24.04 LTS или минималистичный дистрибутив Alpine Linux в контейнере)**. Выбор обусловлен минимальными накладными расходами на оперативную память и процессор (работа в *headless-режиме* без графического интерфейса), высочайшей надежностью и аптаймом, проверенной моделью разграничения прав доступа ядра (SELinux, cgroups) и нативной поддержкой контейнеризации.

3. Анализ путей внутри проекта

В структуре спроектированного сервиса выделены три типа путей:

1. Абсолютный путь:

```
/home/developer/projects/rest-api-service/app/main.py
```

Характеристика: Полный неизменяемый адрес файла от корневого каталога файловой системы (/). Однозначно адресует ресурс в ОС независимо от текущего рабочего каталога. В исходном коде при-

ложения абсолютные пути категорически запрещены, так как они делают проект неработоспособным на компьютерах других разработчиков и на серверах.

2. Относительный путь № 1 (от корня проекта к скрипту миграции):

```
./migrations/versions/001_initial_tables.py
```

Характеристика: Путь строится относительно корневого каталога репозитория (. /). Он переносим и гарантирует точное исполнение команды миграций (`alembic upgrade head`) в любом окружении.

3. Относительный путь № 2 (с выходом в родительский каталог):

```
../app/core/config.py
```

Характеристика: Путь указан относительно директории тестов `tests/`. Использование маркера `../` указывает на подъем на один уровень вверх в корневой каталог с последующим обращением к модулю конфигурации. Это гарантирует изолированный импорт параметров ядра в тестах без привязки к глобальному дереву ФС.

4. Анализ и исправление «плохих» имен файлов

Исходное имя	Найденные ошибки	Исправленное имя	Техническое обоснование
Мои Файлы/Главный Скрипт.py	1. Кириллица в пути и имени. 2. Пробелы между словами. 3. Верхний регистр букв.	app/main.py (или scripts/ seed_database.py)	Пробелы воспринимаются шеллом как разделители параметров команды. Кириллица ломает локаль и кодировку на Linux-сервере. В REST API точка входа по стандарту именуется main.py внутри папки app/.
C:\temp\Data.CSV	1. Абсолютный путь Windows (C:\temp). 2. Обратные слеш (\). 3. Заглавные буквы. 4. Расширение .CSV в верхнем регистре.	data/ initial_data.csv	Каталог C:\temp отсутствует на Linux-серверах. Linux чувствителен к регистру (case-sensitive), поэтому расширение .CSV не распознается скриптами, ожидающими .csv. Путь преобразован в относительный.
тест 1.js	1. Кириллица в имени. 2. Пробел перед цифрой. 3. Неверное расширение (.js вместо .py).	tests/ test_auth.py (или tests/ test_api.py)	Тесты бэкенда на Python пишутся на Python (.py). Пробелы и кириллица нарушают автоматическое обнаружение тестов утилитой pytest. Фреймворк требует маску test_*.py в каталоге tests/.

5. Продвинутая часть: Единое окружение команды

Конфигурация среды

Для гарантированной синхронизации среды разработки во всей команде развертывается стек:

1. **Docker и Docker Compose:** Приложение, база данных PostgreSQL и вспомогательные сервисы описываются декларативно в файлах Dockerfile и docker-compose.yml.
2. **Подсистема WSL2 (Ubuntu):** Члены команды, использующие Windows, запускают рабочие контейнеры и терминал внутри WSL2, что исключает конфликт системных вызовов.
3. **Фиксация зависимостей:** Использование манифеста pyproject.toml со строгой фиксацией хэшей и версий библиотек.

Почему это навсегда устраняет баг «работает только у меня»

Баг «works on my machine» обусловлен фундаментальными межплатформенными различиями:

1. **Регистрозависимость файловых систем:** Файловая система Windows (NTFS) не различает регистр букв: путь `data/Users.csv` успешно откроет файл `data/users.csv`. Однако файловая система Linux (ext4) строго учитывает регистр, что приведет к фатальной ошибке `FileNotFoundError` на сервере. Контейнер гарантирует исполнение в среде ext4 независимо от ОС разработчика.
2. **Переводы строк (CRLF vs LF):** Windows использует возврат каретки и перевод строки `\r\n` (CRLF), тогда как Linux — только перевод строки `\n` (LF). Скрипты инициализации с CRLF в Linux завершаются ошибкой `/bin/bash^M: bad interpreter`. Контейнеризация и строгий `.gitattributes` исключают подобные сбои.
3. **Разница системных пакетов и версий:** Отличия в минорных версиях Python (3.10 против 3.12) или версиях C-библиотек (`libpq-dev` для работы с PostgreSQL) вызывают трудноуловимые сбои компиляции. Docker-образ фиксирует точную версию интерпретатора и всех системных библиотек.
4. **Полная воспроизводимость:** Контейнер инкапсулирует приложение со всей его инфраструктурой. Команда `docker compose up --build` создает абсолютно идентичное рабочее окружение на любой машине.

Заключение

В рамках работы спроектирована эталонная структура REST API-сервиса, удовлетворяющая критериям отказоустойчивости, информационной безопасности и переносимости. Освоены правила формирования путей и доказана эффективность контейнеризации в устранении платформозависимых дефектов.